

Deployment — Docker, AWS, CI/CD

Infrastructure: AWS (ECS/EKS, RDS, ElastiCache, S3, ALB, CloudFront)

Containers: Docker + Docker Compose (dev) → ECS Fargate (prod) **CI/CD:** GitHub Actions

Table of Contents

1. [Docker Setup](#)
 2. [Docker Compose \(Development\)](#)
 3. [Production Infrastructure \(AWS\)](#)
 4. [Nginx Configuration](#)
 5. [Environment Variables \(Production\)](#)
 6. [GitHub Actions CI/CD](#)
 7. [Database Setup \(RDS\)](#)
 8. [Redis Setup \(ElastiCache\)](#)
 9. [Deployment Checklist](#)
-

1. Docker Setup

Dockerfile

FROM python:3.12-slim-bookworm **AS** base

Install system dependencies (PostGIS GDAL libraries required)

RUN apt-get update && apt-get install -y \
gdal-bin \
libgdal-dev \
libpq-dev \
gcc \
g++ \
curl \
&& rm -rf /var/lib/apt/lists/*

ENV PYTHONUNBUFFERED=1

ENV PYTHONDONTWRITEBYTECODE=1

ENV GDAL_LIBRARY_PATH=/usr/lib/x86_64-linux-gnu/libgdal.so

WORKDIR /app

```

COPY requirements/base.txt requirements/production.txt ./
    requirements/

# Install Python dependencies
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements/production.txt

COPY . .

# Collect static files
RUN python manage.py collectstatic --noinput

# Create non-root user for security
RUN addgroup --system appuser && adduser --system --group appuser
USER appuser

EXPOSE 8000

# Default command: Daphne ASGI server
CMD ["daphne", "-b", "0.0.0.0", "-p", "8000", "--proxy-headers",
    "config.asgi:application"]

# Dockerfile.celery – for Celery worker and beat containers
FROM python:3.12-slim-bookworm

RUN apt-get update && apt-get install -y \
    gdal-bin libgdal-dev libpq-dev gcc \
    && rm -rf /var/lib/apt/lists/*

ENV PYTHONUNBUFFERED=1
WORKDIR /app

COPY requirements/base.txt requirements/production.txt ./
    requirements/
RUN pip install --no-cache-dir -r requirements/production.txt

COPY . .

# CMD overridden in docker-compose / ECS task definition
CMD ["celery", "-A", "celery_app", "worker", "-Q", "default", "-c", "4"]

```

2. Docker Compose (Development)

```

# docker-compose.yml

version: '3.9'

services:

```

PostgreSQL with PostGIS extension

```
db:
  image: postgis/postgis:16-3.4-alpine
  volumes:
    - postgres_data:/var/lib/postgresql/data
  environment:
    POSTGRES_DB: bsecure
    POSTGRES_USER: bsecure
    POSTGRES_PASSWORD: dev_password
  ports:
    - "5432:5432"
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U bsecure"]
    interval: 10s
    timeout: 5s
    retries: 5
```

Redis (Channels + Celery broker + Cache)

```
redis:
  image: redis:7-alpine
  ports:
    - "6379:6379"
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5
```

Django API (ASGI with Daphne)

```
api:
  build:
    context: .
    dockerfile: Dockerfile
  command: daphne -b 0.0.0.0 -p 8000 config.asgi:application
  volumes:
    - ./app # Hot reload in development
    - media_files:/app/media # Local media storage
  ports:
    - "8000:8000"
  env_file: .env
  environment:
    DJANGO_SETTINGS_MODULE: config.settings.development
  depends_on:
    db:
      condition: service_healthy
    redis:
      condition: service_healthy
```

Celery Worker (high priority)

```
celery_high:
  build:
    context: .
    dockerfile: Dockerfile.celery
```

```

command: celery -A celery_app worker -Q high_priority -c 2 -n
        worker-high@%h -l info
volumes:
  - ./app
env_file: .env
environment:
  DJANGO_SETTINGS_MODULE: config.settings.development
depends_on:
  - db
  - redis

# Celery Worker (default + low)
celery_default:
  build:
    context: .
    dockerfile: Dockerfile.celery
  command: celery -A celery_app worker -Q default,low_priority
          -c 4 -n worker-default@%h -l info
  volumes:
    - ./app
  env_file: .env
  environment:
    DJANGO_SETTINGS_MODULE: config.settings.development
  depends_on:
    - db
    - redis

# Celery Beat (periodic tasks)
celery_beat:
  build:
    context: .
    dockerfile: Dockerfile.celery
  command: celery -A celery_app beat -l info --scheduler
          django_celery_beat.schedulers:DatabaseScheduler
  volumes:
    - ./app
  env_file: .env
  environment:
    DJANGO_SETTINGS_MODULE: config.settings.development
  depends_on:
    - db
    - redis

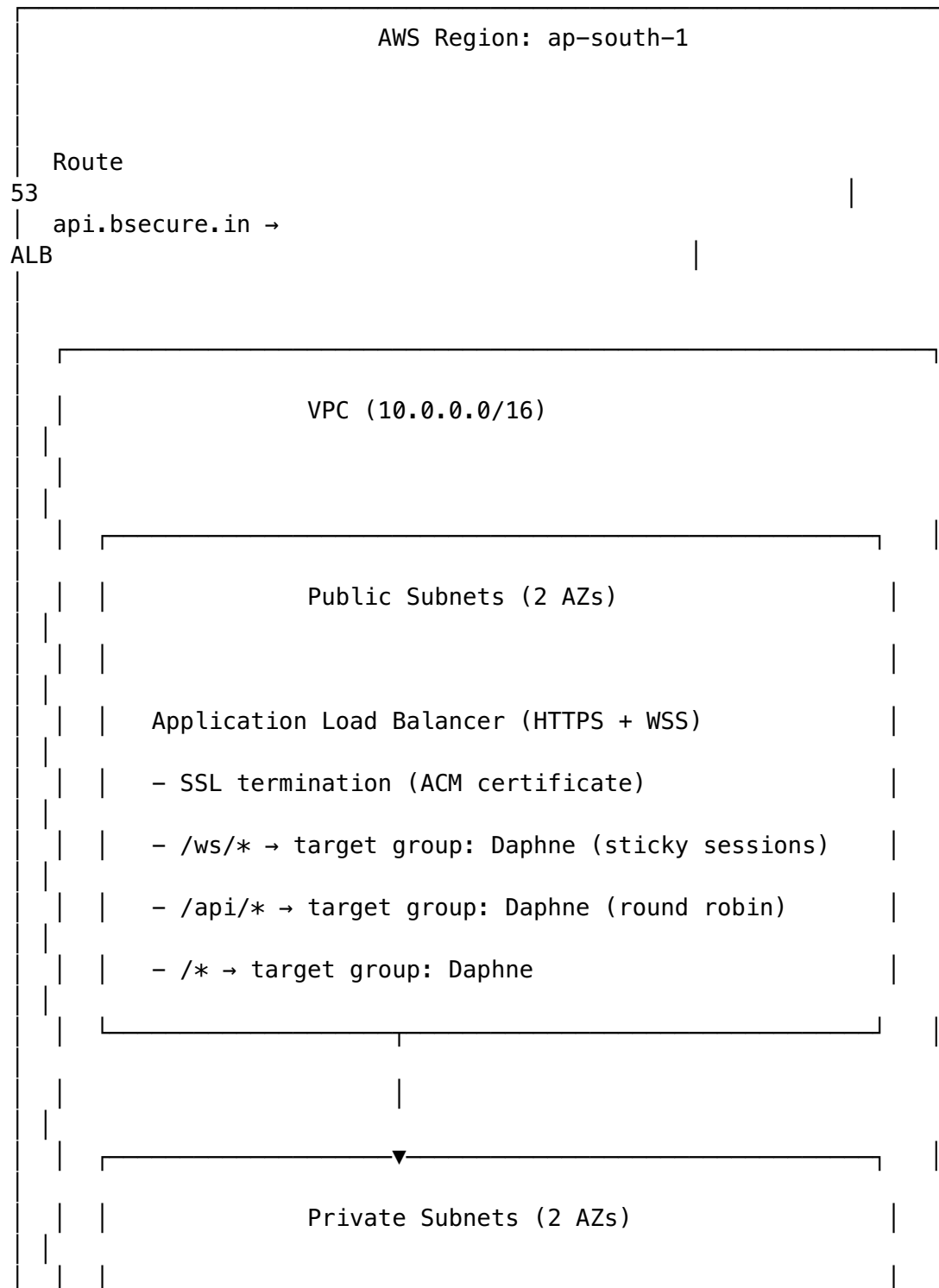
# Flower (Celery monitoring UI)
flower:
  build:
    context: .
    dockerfile: Dockerfile.celery
  command: celery -A celery_app flower --port=5555 --
          basic_auth=admin:admin
  ports:
    - "5555:5555"
  env_file: .env

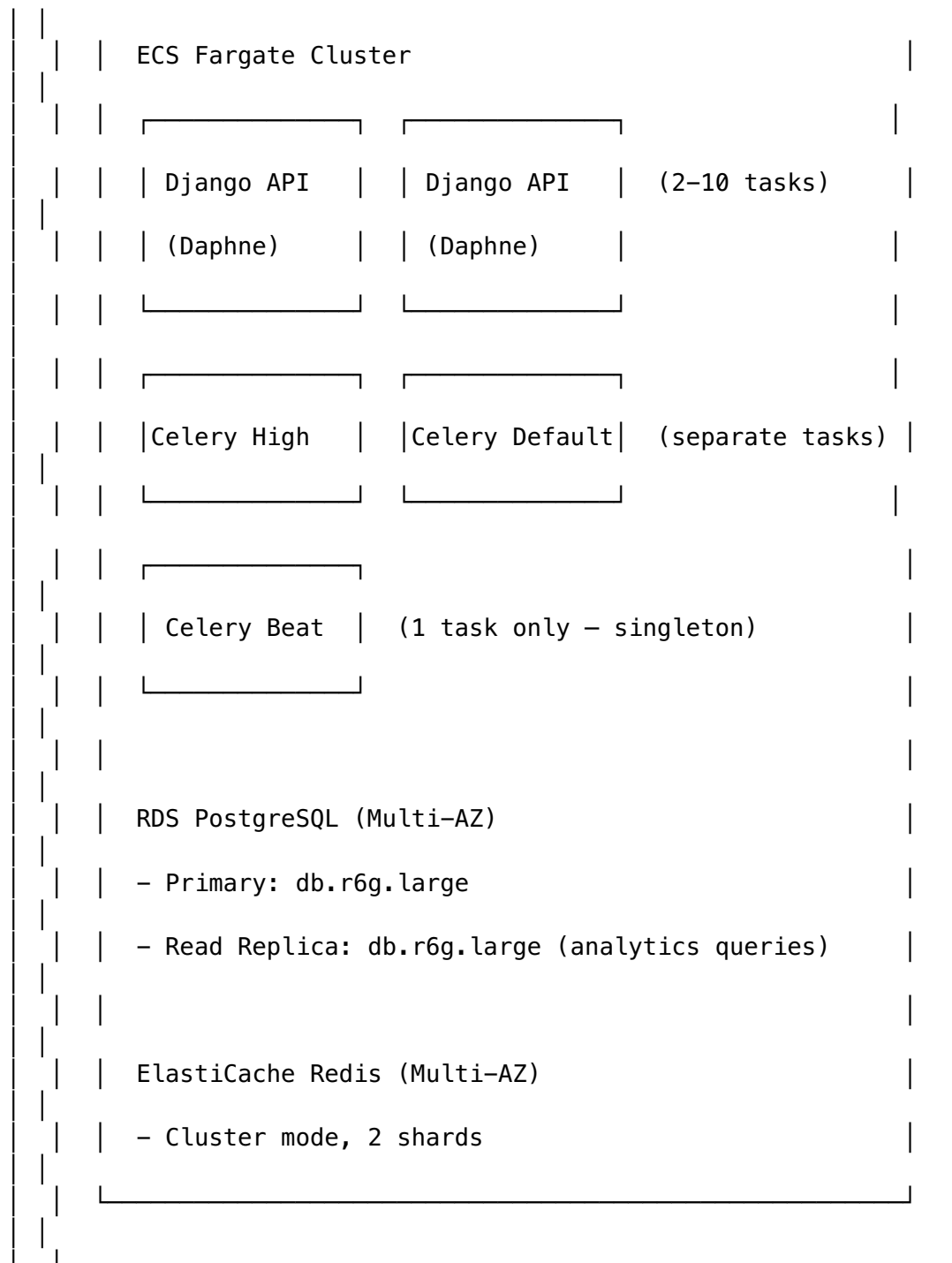
```

```
depends_on:
  - redis

volumes:
  postgres_data:
  media_files:
```

3. Production Infrastructure (AWS)





S3

Buckets:

- bsecure-media-prod (private) – user docs, photos, invoices
- bsecure-static-prod (public) – static files

CloudFront → S3 (static assets
CDN)

ECR (Elastic Container Registry) – Docker image storage

Secrets Manager – all env vars (DB password, API keys, etc.)

CloudWatch – logs, metrics,
alarms

ECS Task Definitions

// api-task-definition.json (simplified)

```
{
  "family": "bsecure-api",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "1024",
  "memory": "2048",
  "containerDefinitions": [
    {
      "name": "api",
      "image": "123456789.dkr.ecr.ap-south-1.amazonaws.com/
bsecure-api:latest",
      "portMappings": [{"containerPort": 8000, "protocol":
"tcp"}],
      "environment": [
        {"name": "DJANGO_SETTINGS_MODULE", "value":
"config.settings.production"}
      ],
      "secrets": [
        {"name": "SECRET_KEY", "valueFrom":
"arn:aws:secretsmanager:..."},
        {"name": "DB_PASSWORD", "valueFrom":
"arn:aws:secretsmanager:..."},
        {"name": "REDIS_URL", "valueFrom":
"arn:aws:secretsmanager:..."}
      ],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/bsecure-api",
          "awslogs-region": "ap-south-1",
          "awslogs-stream-prefix": "api"
        }
      },
      "healthCheck": {
        "command": ["CMD-SHELL", "curl -f http://
localhost:8000/api/health/ || exit 1"],
        "interval": 30,
```

```

        "timeout": 5,
        "retries": 3
    }
}
]
}

```

4. Nginx Configuration

```

# nginx/nginx.conf

upstream daphne_pool {
    least_conn;
    server api1:8000 weight=1;
    server api2:8000 weight=1;
    server api3:8000 weight=1;
    keepalive 32;
}

server {
    listen 80;
    server_name api.bsecure.in;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name api.bsecure.in;

    # SSL (managed by ACM + ALB in production – Nginx is behind
    ALB)
    ssl_certificate /etc/nginx/ssl/cert.pem;
    ssl_certificate_key /etc/nginx/ssl/key.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-
    GCM-SHA256;
    ssl_prefer_server_ciphers off;

    # Security headers
    add_header Strict-Transport-Security "max-age=31536000;
    includeSubDomains; preload" always;
    add_header X-Content-Type-Options nosniff always;
    add_header X-Frame-Options DENY always;
    add_header X-XSS-Protection "1; mode=block" always;

    # Client body size (for document uploads)
    client_max_body_size 25M;

    # WebSocket connections (long timeout)
    location /ws/ {
        proxy_pass http://daphne_pool;
    }
}

```



```

        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 3600s;
        proxy_send_timeout 3600s;
    }

    # API requests
    location /api/ {
        proxy_pass http://daphne_pool;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 30s;
        proxy_send_timeout 30s;
    }

    # Rate limiting for auth endpoints
    location /api/auth/send-otp/ {
        limit_req zone=otp_zone burst=5 nodelay;
        proxy_pass http://daphne_pool;
        proxy_set_header Host $host;
    }

    # Health check (no logging)
    location /api/health/ {
        proxy_pass http://daphne_pool;
        access_log off;
    }
}

# Rate limiting zones
limit_req_zone $binary_remote_addr zone=otp_zone:10m rate=3r/m;
limit_req_zone $binary_remote_addr zone=api_zone:10m rate=100r/m;

```

5. Environment Variables (Production)

All production secrets are stored in **AWS Secrets Manager** (not in environment files). ECS injects them as environment variables at container start.

Fetched from AWS Secrets Manager at runtime:

```
DJANGO_SETTINGS_MODULE=config.settings.production
SECRET_KEY=<50+ char random string>
DEBUG=False
ALLOWED_HOSTS=api.bsecure.in

DB_NAME=bsecure_prod
DB_USER=bsecure_prod
DB_PASSWORD=<fetched from Secrets Manager>
DB_HOST=bsecure-prod.cluster-xyz.ap-south-1.rds.amazonaws.com
DB_PORT=5432

REDIS_URL=rediss://bsecure-prod.xyz.cache.amazonaws.com:6379/0

AWS_ACCESS_KEY_ID=<IAM role – not needed if using ECS task role>
AWS_S3_BUCKET_NAME=bsecure-media-prod
AWS_S3_REGION_NAME=ap-south-1

JWT_ACCESS_TOKEN_LIFETIME_MINUTES=15
JWT_REFRESH_TOKEN_LIFETIME_DAYS=30

FIREBASE_SERVICE_ACCOUNT_KEY_PATH=/run/secrets/firebase_key.json

TWILIO_ACCOUNT_SID=ACxxxxxxxxxx
TWILIO_AUTH_TOKEN=<from Secrets Manager>
TWILIO_PHONE_NUMBER=+1234567890

RAZORPAY_KEY_ID=rzp_livexxxxxxxx
RAZORPAY_KEY_SECRET=<from Secrets Manager>
RAZORPAY_WEBHOOK_SECRET=<from Secrets Manager>

STRIPE_SECRET_KEY=sk_livexxxxxxxx
STRIPE_WEBHOOK_SECRET=whsec_xxxxxxxxxx
STRIPE_PUBLISHABLE_KEY=pk_live_xxxxxxxxxx

SENDGRID_API_KEY=SG.xxxxxxxxxx
DEFAULT_FROM_EMAIL=noreply@bsecure.in

GOOGLE_MAPS_API_KEY=AIzaxxxxxxxxxxxxxxxxxxxxx
SENTRY_DSN=https://xxxxx@sentry.io/xxxxx

CORS_ALLOWED_ORIGINS=https://admin.bsecure.in

COMPANY_GSTIN=29AAAAA0000A1Z5
SOS_SUPERVISOR_PHONES=+919876543210,+919876543211
ONCALL_ENGINEER_EMAIL=oncall@bsecure.in
```

6. GitHub Actions CI/CD

```
# .github/workflows/ci.yml
```

```

name: CI – Test & Lint

on:
  pull_request:
    branches: [main, develop]

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgis/postgis:16-3.4
        env:
          POSTGRES_DB: bsecure_test
          POSTGRES_USER: bsecure
          POSTGRES_PASSWORD: test_password
        ports: ["5432:5432"]
        options: --health-cmd pg_isready --health-interval 10s --
          health-retries 5

      redis:
        image: redis:7-alpine
        ports: ["6379:6379"]
        options: --health-cmd "redis-cli ping" --health-interval
          10s

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python 3.12
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: 'pip'

      - name: Install system dependencies
        run: |
          sudo apt-get update
          sudo apt-get install -y gdal-bin libgdal-dev libpq-dev

      - name: Install Python dependencies
        run: |
          pip install -r requirements/development.txt

      - name: Run linting
        run: |
          black --check .
          isort --check-only .
          flake8 .

      - name: Run security scan
        run: |

```

```
bandit -r apps/ -ll
safety check -r requirements/base.txt
```

- name: Run tests
env:
 DJANGO_SETTINGS_MODULE: config.settings.development
 DB_NAME: bsecure_test
 DB_USER: bsecure
 DB_PASSWORD: test_password
 DB_HOST: localhost
 REDIS_URL: redis://localhost:6379/0
 SECRET_KEY: test-secret-key-not-for-production
run: |
 python manage.py migrate
 pytest --cov=apps --cov-report=xml --cov-fail-under=75
 -v
- name: Upload coverage to Codecov
uses: codecov/codecov-action@v4
with:
 file: coverage.xml

```
# .github/workflows/deploy.yml
```

```
name: Deploy to Production
```

```
on:
```

```
  push:  
    branches: [main]
```

```
jobs:
```

```
  deploy:  
    runs-on: ubuntu-latest  
    environment: production # Requires manual approval in GitHub
```

```
  steps:
```

- uses: actions/checkout@v4
- name: Configure AWS credentials
uses: aws-actions/configure-aws-credentials@v4
with:
 aws-access-key-id: \${ secrets.AWS_ACCESS_KEY_ID }
 aws-secret-access-key: \$
 { secrets.AWS_SECRET_ACCESS_KEY }
 aws-region: ap-south-1
- name: Login to Amazon ECR
id: login-ecr
uses: aws-actions/amazon-ecr-login@v2
- name: Build and push Docker image
env:
 ECR_REGISTRY: \${ steps.login-ecr.outputs.registry }

```

    ECR_REPOSITORY: bsecure-api
    IMAGE_TAG: ${github.sha}
run: |
    docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:
$IMAGE_TAG .
    docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:latest .
    docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
    docker push $ECR_REGISTRY/$ECR_REPOSITORY:latest

- name: Run database migrations
run: |
    aws ecs run-task \
        --cluster bsecure-prod \
        --task-definition bsecure-migrate \
        --launch-type FARGATE \
        --network-configuration
"awsvpcConfiguration={subnets=[subnet-
xxx],securityGroups=[sg-xxx]}" \
        --overrides '{"containerOverrides":
[{"name":"api","command":
["python","manage.py","migrate","--no-input"]}]}
'

- name: Deploy API service (rolling update)
env:
    IMAGE_TAG: ${github.sha}
run: |
    # Update ECS service – triggers rolling replacement of
tasks
    aws ecs update-service \
        --cluster bsecure-prod \
        --service bsecure-api \
        --force-new-deployment \
        --task-definition bsecure-api

    # Wait for deployment to stabilize
    aws ecs wait services-stable \
        --cluster bsecure-prod \
        --services bsecure-api

- name: Deploy Celery workers
run: |
    aws ecs update-service --cluster bsecure-prod --service
bsecure-celery-high --force-new-deployment
    aws ecs update-service --cluster bsecure-prod --service
bsecure-celery-default --force-new-deployment
    aws ecs update-service --cluster bsecure-prod --service
bsecure-celery-beat --force-new-deployment

- name: Notify deployment success
if: success()
run: |
    curl -X POST ${ secrets.SLACK_WEBHOOK_URL } \
        -d '{"text": "👍 b-secure backend deployed successfully
- "${github.sha}"}'

```

```
- name: Notify deployment failure
  if: failure()
  run: |
    curl -X POST ${ secrets.SLACK_WEBHOOK_URL } \
      -d '{"text": "b-secure deployment FAILED - '$
        {{ github.sha }}' - check GitHub Actions logs"}'
```

7. Database Setup (RDS)

```
# Initial RDS setup commands (run once via ECS task or bastion
  host)
```

```
# Enable PostGIS extension
```

```
psql -h <rds_host> -U bsecure_prod -d bsecure_prod -c "
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS postgis_topology;
CREATE EXTENSION IF NOT EXISTS pg_trgm; -- For text search
"
```

```
# Run Django migrations
```

```
python manage.py migrate
```

```
# Create superuser (interactive)
```

```
python manage.py createsuperuser
```

```
# Load initial data (guard types, pricing config, etc.)
```

```
python manage.py loaddata fixtures/initial_data.json
```

RDS configuration:

Instance class: db.r6g.large (2 vCPU, 16 GB RAM)

Storage: 200 GB gp3, auto-scaling to 1 TB

Multi-AZ: Yes (automatic failover < 60 seconds)

Backup retention: 35 days

Read replica: 1 (for analytics queries – connect via separate DB URL)

PostgreSQL version: 16

Parameter group: Custom (max_connections=200, shared_buffers=4GB)

8. Redis Setup (ElastiCache)

Engine: Redis 7.x

Mode: Cluster mode enabled (2 shards × 1 replica = 4 nodes)

Node type: cache.r7g.large (2 vCPU, 13 GB)

Multi-AZ: Yes

Encryption: At-rest (AES-256) + In-transit (TLS)

Auth token: Yes (required)

Redis URL format:

rediss:///authtoken@cluster.xyz.cache.amazonaws.com:6379/0

↑ Note: rediss:// (with SSL)

9. Deployment Checklist

Before First Production Deploy

- ☐ RDS instance created with PostGIS extension enabled
- ☐ ElastiCache Redis cluster created
- ☐ S3 buckets created with proper IAM policies
- ☐ ECR repository created
- ☐ All secrets added to AWS Secrets Manager
- ☐ ECS cluster and task definitions created
- ☐ ALB created with HTTPS listener and ACM certificate
- ☐ Security groups configured (API only accepts from ALB, DB only from ECS)
- ☐ IAM roles for ECS tasks (S3 access, Secrets Manager read, ECR pull)
- ☐ Domain configured in Route 53 (api.bsecure.in → ALB)
- ☐ Sentry project created and DSN set in Secrets Manager
- ☐ Firebase service account key uploaded to Secrets Manager
- ☐ Twilio, Razorpay, Stripe webhooks configured with production URLs
- ☐ `python manage.py migrate` run on production DB
- ☐ `python manage.py createsuperuser` run

Health check endpoint returns 200: `curl https://api.bsecure.in/api/health/`

Every Production Deploy

☐

All CI tests passing on the branch

☐

Docker image built and pushed to ECR

☐

DB migrations applied (no destructive migrations without review)

☐

ECS services updated (rolling deploy — zero downtime)

☐

Health check passes after deploy

☐

Sentry error rate not spiking

☐

Monitor CloudWatch for 15 minutes post-deploy